МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Агент

Лабораторийн ажил № 4

Компьютерын ухааны салбар

This e-learning content was developed by the support of KOICA



Дэд профессор, док (PhD), Д.Золзаяа /email: zolzaya@must.edu.mn/
Компьютерын ухааны салбар

Лабораторийн дасгал

Энгийн дүрмийн дагуу шийдвэр гаргадаг агент үүсгэж, хэрэглэгчийн өгөгдөлд үндэслэн хариу үйлдэл үзүүлэх.

Ажлын алхмууд:

- Энгийн дүрмүүдийг тодорхойлох: Агентын шийдвэр гаргалтын нөхцөлүүдийг бичнэ (жишээ нь: цаг агаарын нөхцөл).
- Код бичих: Python ашиглан дүрмийн суурьтай агент үүсгэнэ.
- Турших: Олон төрлийн өгөгдөл оруулан агентын хариу үйлдлийг шалгана.



Лабораторийн дасгал

```
class WeatherAgent:
    def respond(self, weather):
    if weather.lower() == "rainy":
        return "Борооны шүхэр аваарай!"
    elif weather.lower() == "sunny":
        return "Нарны шилээ мартуузай!"
    elif weather.lower() == "snowy":
        return "Дулаан хувцас өмсөөрэй!"
    else:
        return "Цаг агаарын мэдээлэл тодорхойгүй байна."

# Агенттай харилцах
agent = WeatherAgent()
user_input = input("Цаг агаар ямар байна? (rainy, sunny, snowy): ")
print("Agent: ", agent.respond(user_input))
```



Лабораторийн дасгал

Орчны өгөгдлийг боловсруулан шийдвэр гаргадаг агент хөгжүүлэх.

```
class EnvironmentAgent:
        def __init__(self, pollution_level):
        self.pollution_level = pollution_level

        def act(self):
        if self.pollution_level > 70:
        return "Бохирдол өндөр байна. Агаар шүүгч асаана уу."
        elif self.pollution_level > 30:
        return "Бохирдол дунд зэрэг байна. Цонхоо хаана уу."
        else:
        return "Бохирдол бага байна. Бүх зүйл хэвийн."

# Туршилт
pollution_level = int(input("Орчны бохирдлын түвшин (0-100): "))
agent = EnvironmentAgent(pollution_level)
print("Agent: ", agent.act())
```



Лабораторийн дасгал

Multi-modal agent-уудын код нь ихэвчлэн TensorFlow, PyTorch зэрэг машин сургалтын санг ашиглан хөгжүүлдэг бөгөөд текст, дүрс, дуу хоолой гэх мэт олон төрлийн өгөгдөлтэй ажиллах чадвартай загваруудыг бий болгодог. Доорх жишээнд Hugging Face Transformers санг ашиглан текст болон дүрсний өгөгдлийг боловсруулж, multi-modal агентын эх кодыг харуулъя.

https://huggingface.co/



Лабораторийн дасгал

- VisionEncoderDecoderModel: Дүрсийг таньж, текст үүсгэхэд ашигладаг. Hugging Face-ийн ViT (Vision Transformer) болон GPT2 (Generative Pre-trained Transformer 2)-ийг хослуулсан загвар.
- ViTImageProcessor: Дүрсийг урьдчилан боловсруулж, загварт тохирсон тензорт хувиргана.
- AutoTokenizer: Загвараас гарсан тоон үр дүнг уншигдах текст болгон хувиргадаг.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

Тухайн төхөөрөмжийг GPU ашиглаж чадах эсэхийг шалгаж, боломжтой бол ашиглана. Хэрэв GPU бай
бол CPU дээр ажиллана.



Лабораторийн дасгал

```
image_path = "sample_image.jpg"  # Өөрийн дүрсний замыг оруулна уу
image = Image.open(image_path)

inputs = processor(images=image, return_tensors="pt").to(device)
```

image.open(image_path): PIL санг ашиглан дүрсийг уншина.

processor: Уг дүрсийг PyTorch тензорт хөрвүүлж, загварт оруулахад бэлдэнэ.



Лабораторийн дасгал

```
output_ids = model.generate(inputs["pixel_values"], max_length=16, num_beams=4)
captions = tokenizer.decode(output_ids[0], skip_special_tokens=True)
print(f"Generated Caption: {captions}")
```

model.generate: Загварын ачаар дүрсийн өгөгдлийг ашиглан текстэн тайлбар үүсгэнэ.

- max_length=16: Текстийн хамгийн их уртыг 16 тэмдэгтээр хязгаарласан.
- num_beams=4: Beam search алгоритмыг ашиглан гарцыг илүү оновчтой болгоно.

tokenizer.decode: Загварын гарцыг уншигдахуйц текст болгон хөрвүүлнэ.



Лабораторийн дасгал

```
import torch
from transformers import VisionEncoderDecoderModel, ViTImageProcessor, AutoTokenizer
from PIL import Image

# Загвар болон процессорыг ачаалах
model = VisionEncoderDecoderModel.from_pretrained("nlpconnect/vit-gpt2-image-captioning")
processor = ViTImageProcessor.from_pretrained("nlpconnect/vit-gpt2-image-captioning")
tokenizer = AutoTokenizer.from_pretrained("nlpconnect/vit-gpt2-image-captioning")

# Төхөөрөмжийг тохируулах (GPU эсвэл CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Дүрсний өгөгдөл ачаалах
image_path = "sample_image.jpg"  # Өөрийн дүрсний замыг оруулна уу
image = Image.open(image_path)

# Дүрсийг боловсруулж, текстэн гаралт үүсгэх
inputs = processor(images=image, return_tensors="pt").to(device)
output_ids = model.generate(inputs["pixel_values"], max_length=16, num_beams=4)
captions = tokenizer.decode(output_ids[0], skip_special_tokens=True)

print(f"Generated Caption: {captions}")
```



Лабораторийн удирдамж

Дараах даалгаваруудыг OpenAI API эсвэл Hugging Face Hub дээрх бусад multi-modal загваруудыг ашиглаж сайжруулна уу.

- Текст + Дүрс: Текстэн хүсэлтийг дүрсийн өгөгдөлтэй хослуулан хариу гаргах.
- Дүрс + Дуу хоолой: Дүрсийг таних болон дуу хоолойгоор тайлбар өгөх.
- Хосолсон: ChatGPT эсвэл GPT загваруудтай хослуулж хэрэглэгчтэй харилцах чадварыг сайжруулах.

